Inflation Prediction

Nicolas Aragona

2026-07-21

Contents

The goal of this report is to construct an autoregressive model for monthly inflation in Argentina and forecast the next four months.

1 Packages and settings

```
library(readr)
library(dplyr)
library(ggplot2)
library(forecast)
library(tseries)
library(knitr)

# Input file
data_path <- paste0(
  "C:/Users/narag/Desktop/my little project/",
  "Inflation Prediction ARG/files/",
  "ipc_general_level_series.csv"
)

# Output CSV file
output_path <- paste0(
  "C:/Users/narag/Desktop/my little project/",
  "Inflation Prediction ARG/files/",
  "argentina_inflation_forecast.csv"
)

# Model settings
seasonal_window <- 13
candidate_lags <- 2:5
window_size <- 60
cv_horizon <- 1
final_horizon <- 4

# TRUE is consistent with including a non-zero mean
# when estimating the AR models on the differenced series.
use_drift <- TRUE
```

2 Import and validate the data

```
if (!file.exists(data_path)) {
  stop(
    paste(
      "The input CSV file was not found.",
      "Check the value assigned to data_path."
    )
  )
}

inflation_raw <- read_csv(
  data_path,
  show_col_types = FALSE
)
```

```r
required_columns <- c(
  "date",
  "monthly_inflation_pct"
)

if (!all(required_columns %in% names(inflation_raw))) {
  stop(
    paste(
      "The CSV must contain the columns:",
      paste(required_columns, collapse = ", ")
    )
  )
}

inflation <- inflation_raw %>%
  transmute(
    date = as.Date(date),
    monthly_inflation_pct =
      as.numeric(monthly_inflation_pct)
  ) %>%
  mutate(
    # Normalize every observation to the first day of its month.
    date = as.Date(
      format(date, "%Y-%m-01")
    )
  ) %>%
  arrange(date)

if (anyNA(inflation$date)) {
  stop("At least one date could not be converted into an R Date.")
}

if (anyNA(inflation$monthly_inflation_pct)) {
  stop("The inflation series contains missing or non-numeric values.")
}

if (anyDuplicated(inflation$date) > 0) {
  stop("The dataset contains duplicate months.")
}

# Verify that months are consecutive.
month_index <- (
  as.integer(format(inflation$date, "%Y")) * 12
) + as.integer(format(inflation$date, "%m"))

if (any(diff(month_index) != 1)) {
  stop(
    paste(
      "The dataset is not a continuous monthly series.",
      "Check for missing months."
    )
  )
}

if (nrow(inflation) <= window_size + 1) {
  stop(
```

```
    paste(
      "There are not enough observations for a",
      window_size,
      "month rolling window."
    )
  )
}

head(inflation)
```

```
## # A tibble: 6 x 2
##   date       monthly_inflation_pct
##   <date>                     <dbl>
## 1 2017-01-01                   1.6
## 2 2017-02-01                   2.1
## 3 2017-03-01                   2.4
## 4 2017-04-01                   2.7
## 5 2017-05-01                   1.4
## 6 2017-06-01                   1.2
```

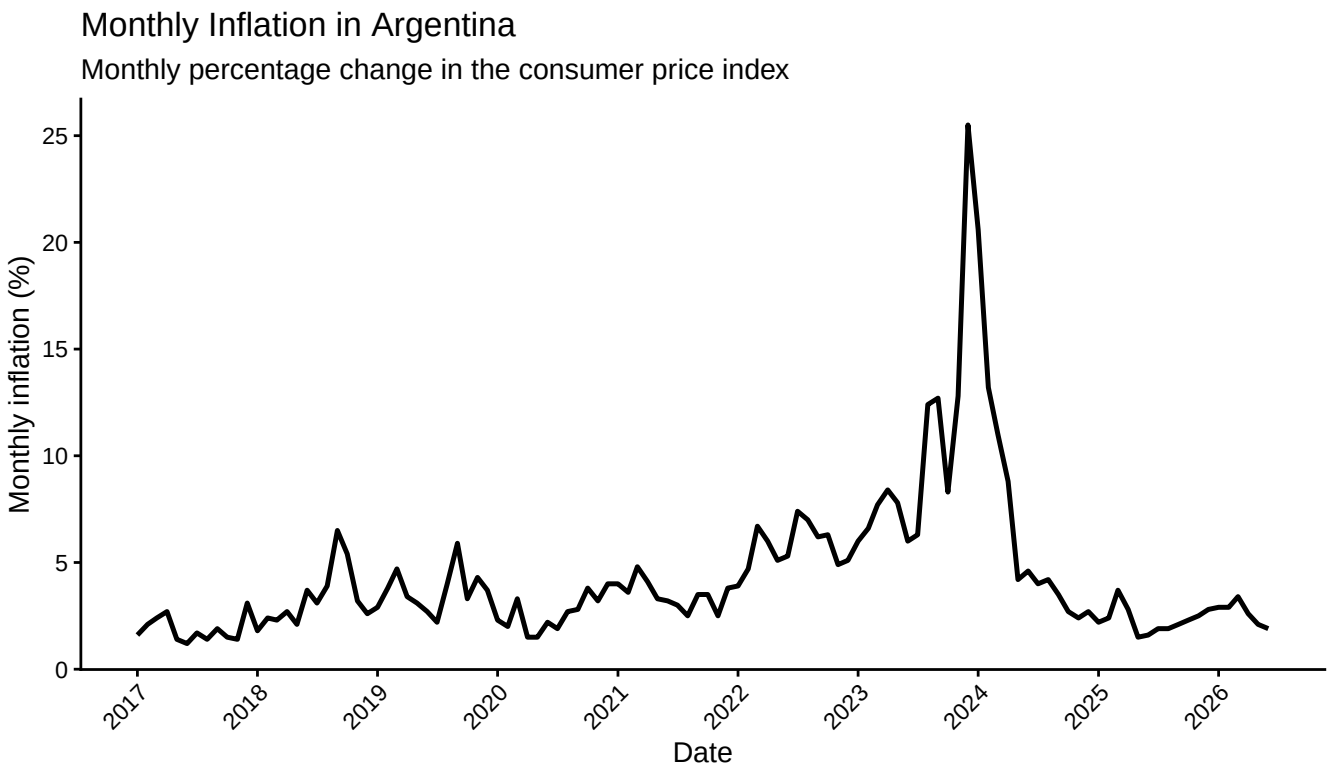
```
tail(inflation)
```

```
## # A tibble: 6 x 2
##   date       monthly_inflation_pct
##   <date>                     <dbl>
## 1 2026-01-01                   2.9
## 2 2026-02-01                   2.9
## 3 2026-03-01                   3.4
## 4 2026-04-01                   2.6
## 5 2026-05-01                   2.1
## 6 2026-06-01                   1.9
```

3 Visualize monthly inflation

```
ggplot(
  inflation,
  aes(
    x = date,
    y = monthly_inflation_pct
  )
) +
  geom_line(linewidth = 0.9) +
  labs(
    title = "Monthly Inflation in Argentina",
    subtitle = "Monthly percentage change in the consumer price index",
    x = "Date",
    y = "Monthly inflation (%)"
  ) +
  scale_x_date(
    date_breaks = "1 year",
    date_labels = "%Y"
  ) +
  theme_classic() +
  theme(
    axis.text.x = element_text(
      angle = 45,
```

```
      hjust = 1
    )
  )
```



4 Convert the data into a monthly time series

```
start_year <- as.integer(
  format(min(inflation$date), "%Y")
)

start_month <- as.integer(
  format(min(inflation$date), "%m")
)

inflation_ts <- ts(
  inflation$monthly_inflation_pct,
  start = c(start_year, start_month),
  frequency = 12
)

inflation_ts
```

```
##       Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
## 2017  1.6  2.1  2.4  2.7  1.4  1.2  1.7  1.4  1.9  1.5  1.4  3.1
## 2018  1.8  2.4  2.3  2.7  2.1  3.7  3.1  3.9  6.5  5.4  3.2  2.6
## 2019  2.9  3.8  4.7  3.4  3.1  2.7  2.2  4.0  5.9  3.3  4.3  3.7
## 2020  2.3  2.0  3.3  1.5  1.5  2.2  1.9  2.7  2.8  3.8  3.2  4.0
## 2021  4.0  3.6  4.8  4.1  3.3  3.2  3.0  2.5  3.5  3.5  2.5  3.8
## 2022  3.9  4.7  6.7  6.0  5.1  5.3  7.4  7.0  6.2  6.3  4.9  5.1
## 2023  6.0  6.6  7.7  8.4  7.8  6.0  6.3 12.4 12.7  8.3 12.8 25.5
```

```
## 2024 20.6 13.2 11.0  8.8  4.2  4.6  4.0  4.2  3.5  2.7  2.4  2.7
## 2025  2.2  2.4  3.7  2.8  1.5  1.6  1.9  1.9  2.1  2.3  2.5  2.8
## 2026  2.9  2.9  3.4  2.6  2.1  1.9
```

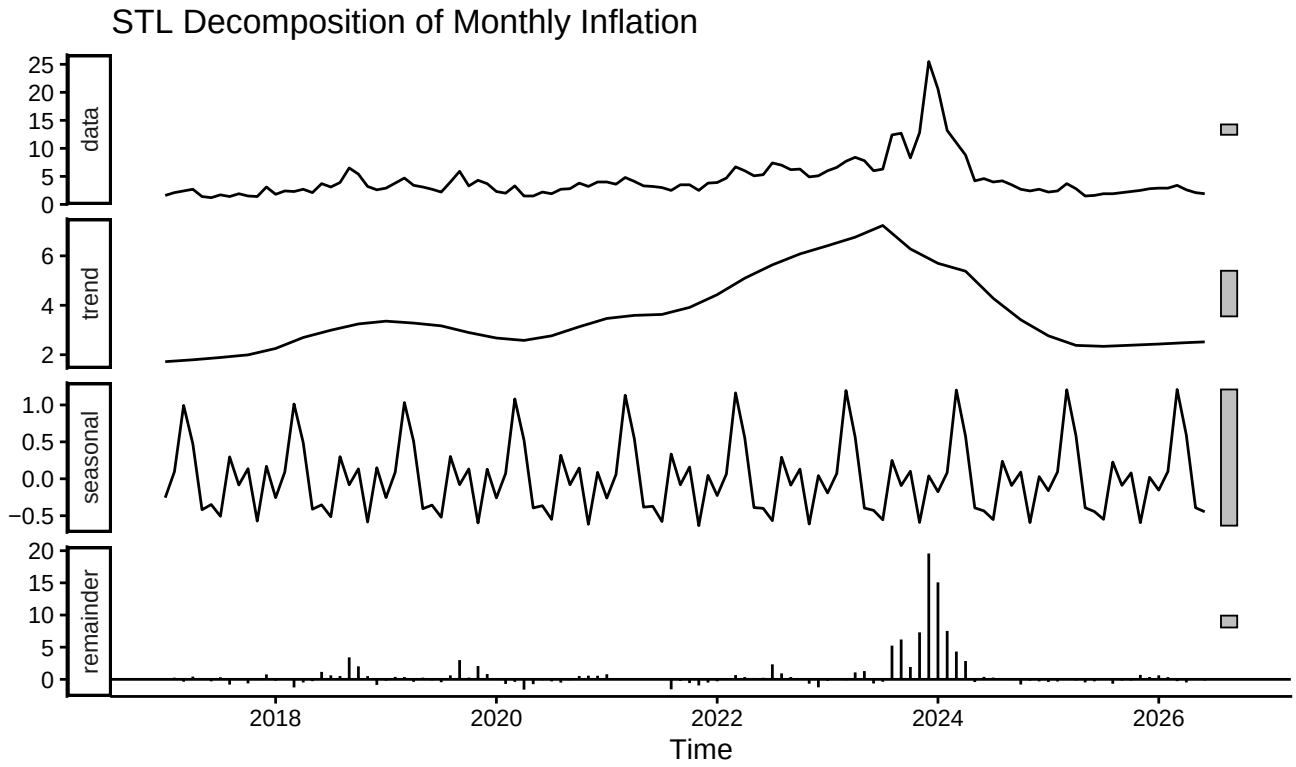
The frequency is set to 12 because the observations are monthly.

5 STL decomposition

The STL decomposition separates observed inflation into trend, seasonal, and remainder components.

```
inflation_stl <- stl(
  inflation_ts,
  s.window = seasonal_window,
  robust = TRUE
)

autoplot(inflation_stl) +
  labs(
    title = "STL Decomposition of Monthly Inflation"
  ) +
  theme_classic()
```

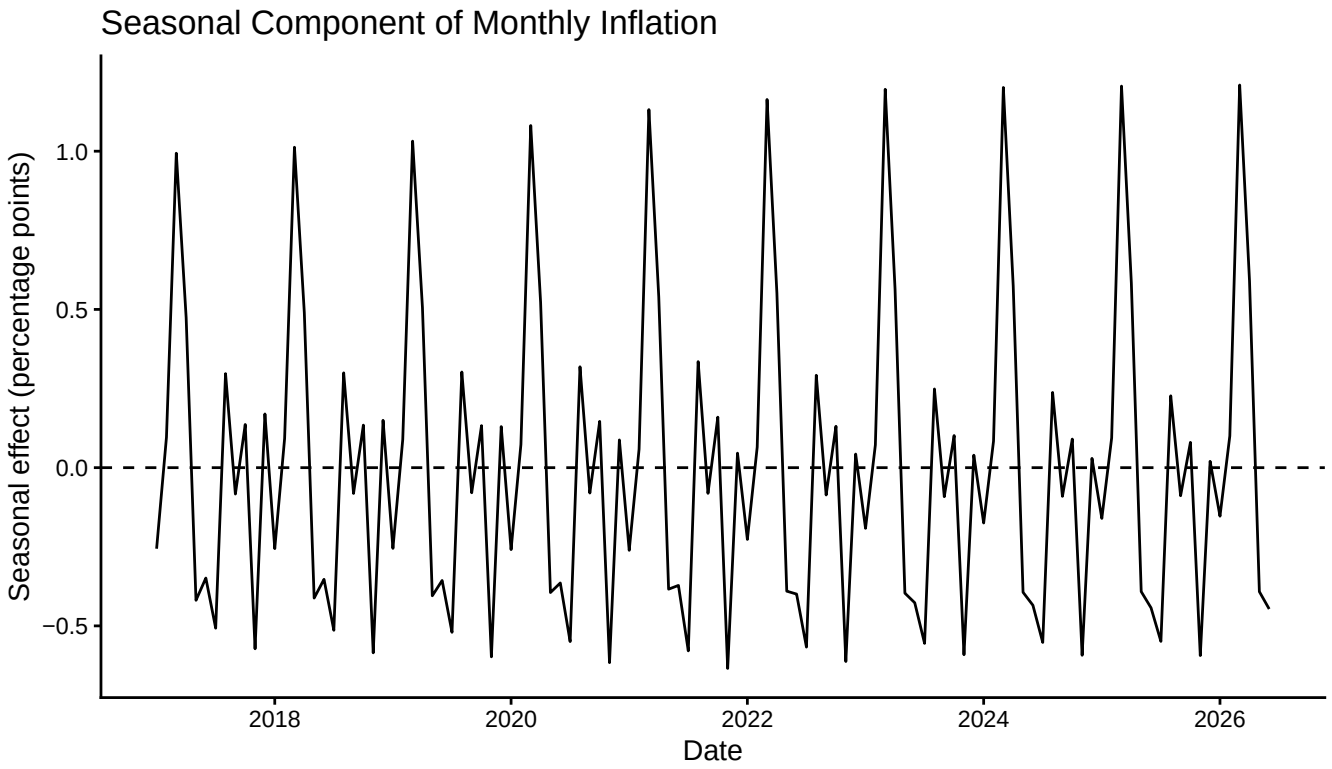


5.1 Seasonal component

```
seasonal_ts <- inflation_stl$time.series[, "seasonal"]

autoplot(seasonal_ts) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
```

```
    title = "Seasonal Component of Monthly Inflation",
    x = "Date",
    y = "Seasonal effect (percentage points)"
  ) +
  theme_classic()
```

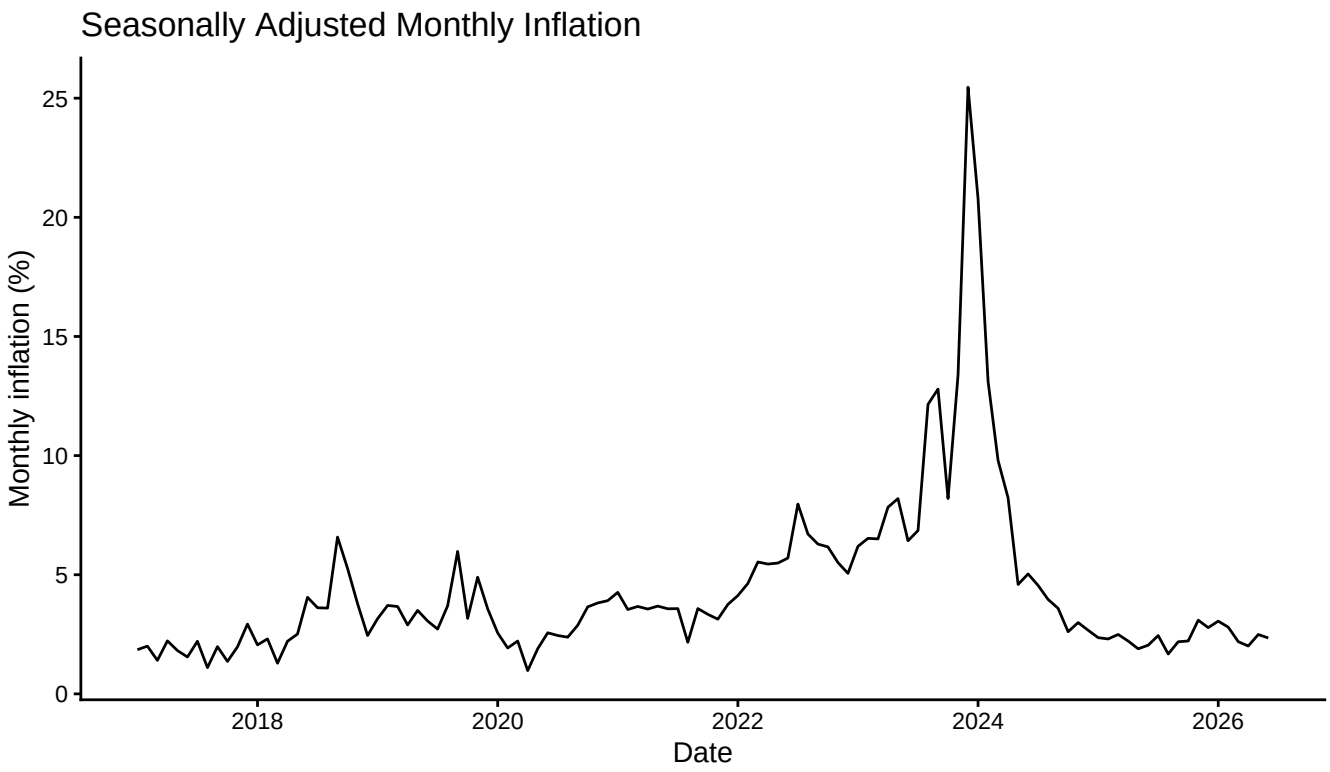


Positive seasonal values increase observed inflation relative to its seasonally adjusted level. Negative values decrease observed inflation.

5.2 Seasonally adjusted inflation

```
inflation_sa <- seasadj(inflation_stl)

autoplot(inflation_sa) +
  labs(
    title = "Seasonally Adjusted Monthly Inflation",
    x = "Date",
    y = "Monthly inflation (%)"
  ) +
  theme_classic()
```



6 Stationarity tests

6.1 Seasonally adjusted inflation in levels

```
adf_level <- adf.test(inflation_sa)

kpss_level <- kpss.test(
  inflation_sa,
  null = "Level"
)

adf_level
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  inflation_sa
## Dickey-Fuller = -2.8118, Lag order = 4, p-value = 0.2396
## alternative hypothesis: stationary
```

```
kpss_level
```

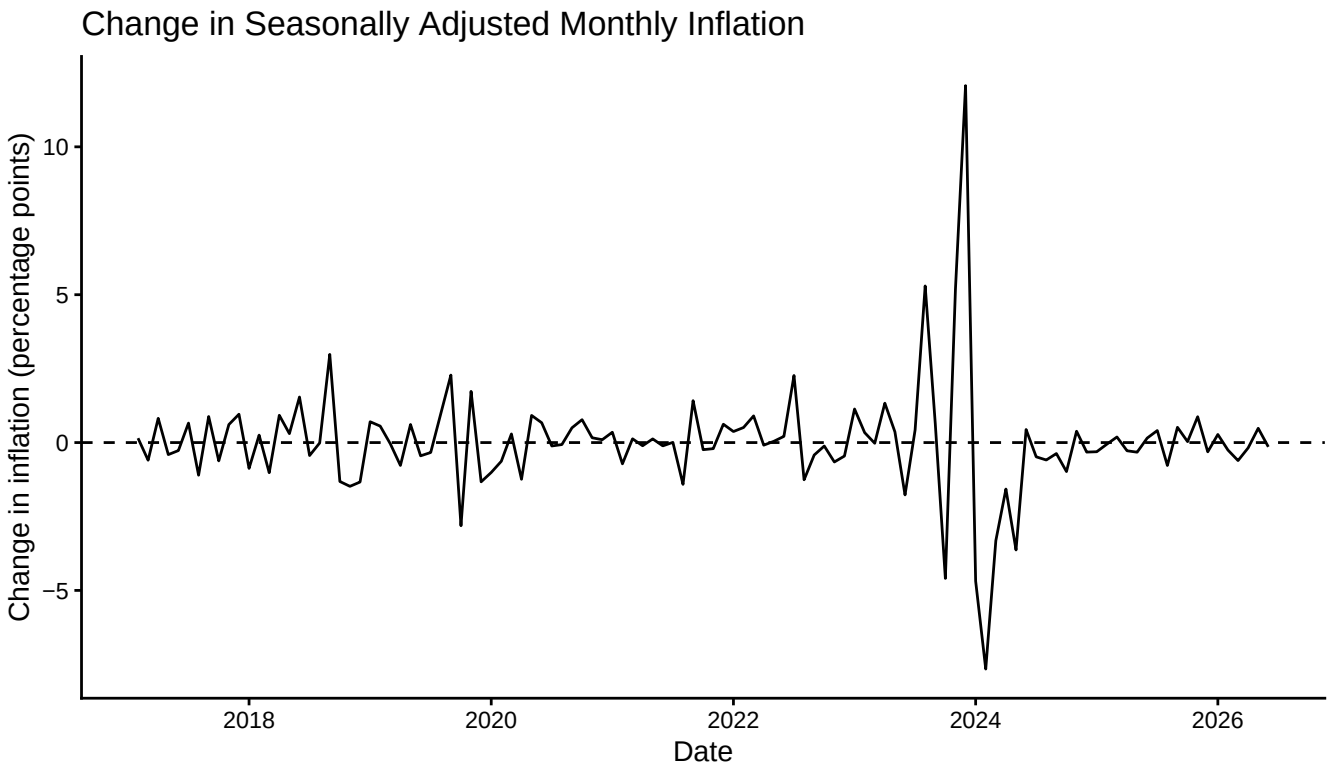
```
##
##  KPSS Test for Level Stationarity
##
## data:  inflation_sa
## KPSS Level = 0.43477, Truncation lag parameter = 4, p-value = 0.06217
```

The stationarity tests indicate that the seasonally adjusted inflation rate in levels is not clearly stationary. The series is therefore first-differenced.

6.2 First difference

```
inflation_sa_diff <- diff(inflation_sa)

autoplot(inflation_sa_diff) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
    title = "Change in Seasonally Adjusted Monthly Inflation",
    x = "Date",
    y = "Change in inflation (percentage points)"
  ) +
  theme_classic()
```



The transformed variable is:

$$\Delta\pi_t^{SA} = \pi_t^{SA} - \pi_{t-1}^{SA}.$$

It represents the monthly change in the seasonally adjusted inflation rate.

```
adf_difference <- adf.test(
  inflation_sa_diff
)

kpss_difference <- kpss.test(
  inflation_sa_diff,
  null = "Level"
)

adf_difference
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  inflation_sa_diff
## Dickey-Fuller = -5.5737, Lag order = 4, p-value = 0.01
## alternative hypothesis: stationary
```

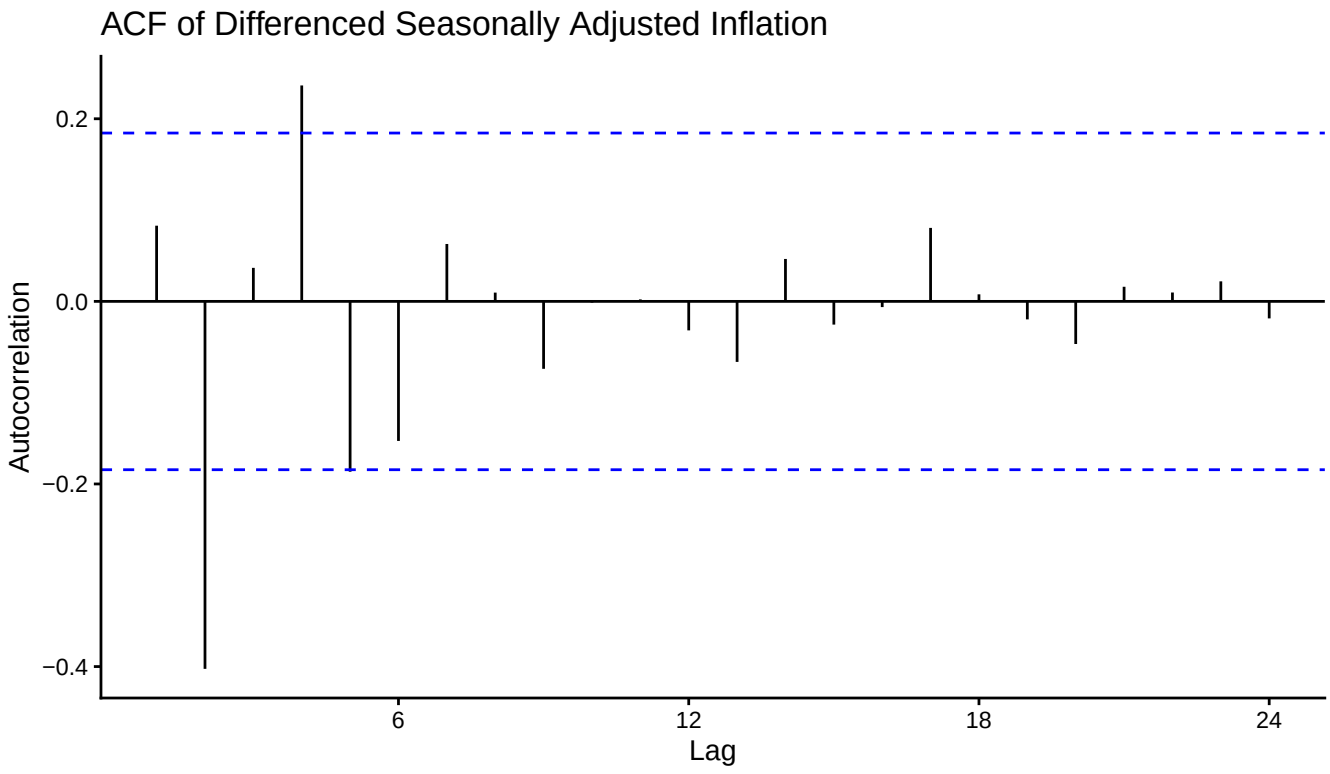
```
kpss_difference
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  inflation_sa_diff
## KPSS Level = 0.054901, Truncation lag parameter = 4, p-value = 0.1
```

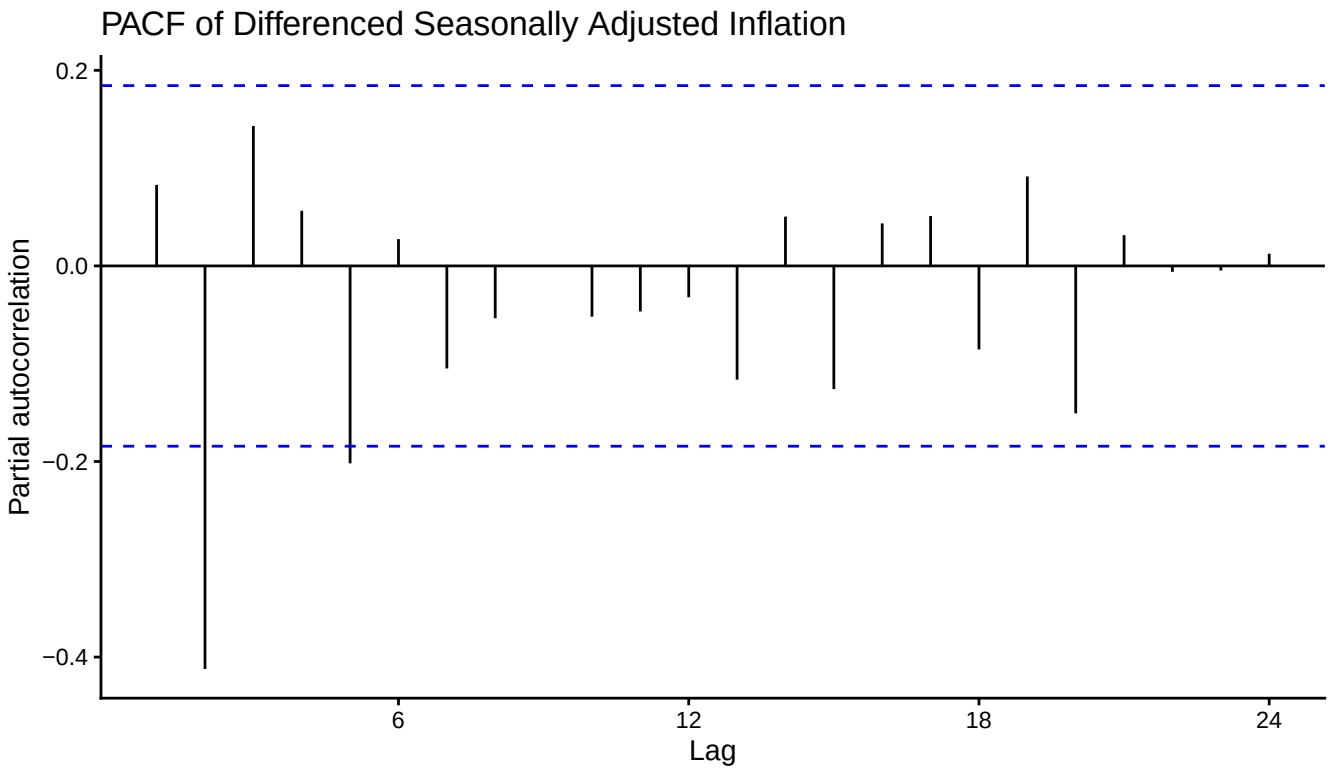
7 Identify candidate AR lags

7.1 ACF and PACF

```
ggAcf(
  inflation_sa_diff,
  lag.max = 24
) +
  labs(
    title = paste(
      "ACF of Differenced",
      "Seasonally Adjusted Inflation"
    ),
    x = "Lag",
    y = "Autocorrelation"
  ) +
  theme_classic()
```



```r
ggPacf(
  inflation_sa_diff,
  lag.max = 24
) +
  labs(
    title = paste(
      "PACF of Differenced",
      "Seasonally Adjusted Inflation"
    ),
    x = "Lag",
    y = "Partial autocorrelation"
  ) +
  theme_classic()
```



The PACF suggests that the second lag is particularly important. However, AR orders from zero through twelve are compared formally.

7.2 Compare AR(0) through AR(12)

```r
all_candidate_lags <- 0:12

all_ar_models <- lapply(
  all_candidate_lags,
  function(p) {
    Arima(
      inflation_sa_diff,
      order = c(p, 0, 0),
      include.mean = TRUE,
      method = "ML"
    )
  }
```

```
)

names(all_ar_models) <- paste0(
  "AR(",
  all_candidate_lags,
  ")"
)

lag_results <- data.frame(
  model = names(all_ar_models),
  lag = all_candidate_lags,
  AICc = sapply(
    all_ar_models,
    function(model) model$aicc
  ),
  BIC = sapply(
    all_ar_models,
    BIC
  )
) %>%
  arrange(BIC)

kable(
  lag_results,
  digits = 3,
  caption = "Information criteria for candidate AR models"
)
```

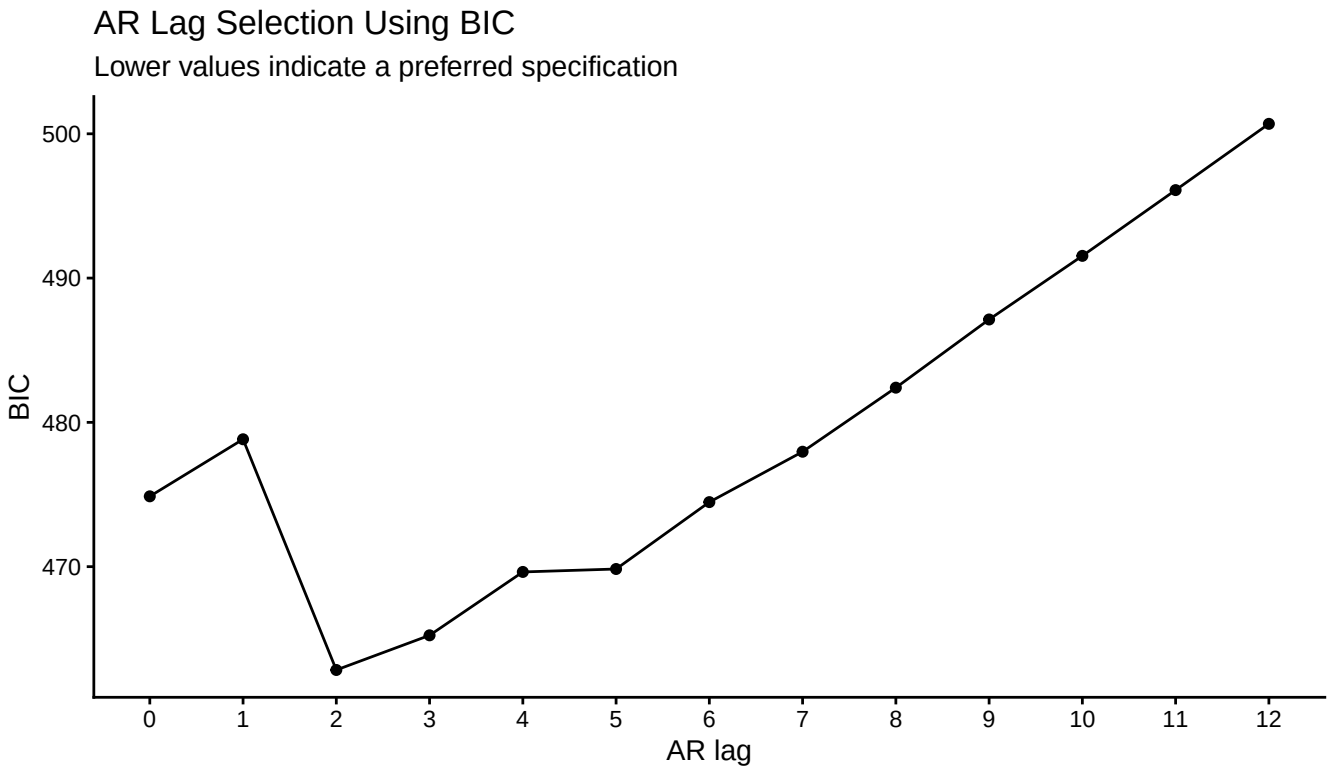
Table 1: Information criteria for candidate AR models

	model	lag	AICc	BIC
AR(2)	AR(2)	2	452.300	462.839
AR(3)	AR(3)	3	452.165	465.241
AR(4)	AR(4)	4	454.062	469.634
AR(5)	AR(5)	5	451.812	469.837
AR(6)	AR(6)	6	454.040	474.474
AR(0)	AR(0)	0	469.528	474.874
AR(7)	AR(7)	7	455.167	477.965
AR(1)	AR(1)	1	470.867	478.829
AR(8)	AR(8)	8	457.288	482.405
AR(9)	AR(9)	9	459.744	487.132
AR(10)	AR(10)	10	461.931	491.540
AR(11)	AR(11)	11	464.318	496.097
AR(12)	AR(12)	12	466.791	500.689

7.3 Plot BIC by lag

```
lag_results %>%
  arrange(lag) %>%
  ggplot(
    aes(
      x = lag,
      y = BIC
    )
  ) +
```

```
  geom_line() +
  geom_point() +
  scale_x_continuous(
    breaks = 0:12
  ) +
  labs(
    title = "AR Lag Selection Using BIC",
    subtitle = "Lower values indicate a preferred specification",
    x = "AR lag",
    y = "BIC"
  ) +
  theme_classic()
```



8 Estimate AR(2), AR(3), AR(4), and AR(5)

The PACF and information criteria suggest retaining AR orders two through five as the main candidate specifications.

```
candidate_models <- lapply(
  candidate_lags,
  function(p) {
    Arima(
      inflation_sa_diff,
      order = c(p, 0, 0),
      include.mean = TRUE,
      method = "ML"
    )
  }
)

names(candidate_models) <- paste0(
  "AR(",
```

```
  candidate_lags,
  ")"
)

candidate_comparison <- data.frame(
  model = names(candidate_models),
  lag = candidate_lags,
  log_likelihood = sapply(
    candidate_models,
    function(model) {
      as.numeric(logLik(model))
    }
  ),
  AICc = sapply(
    candidate_models,
    function(model) model$aicc
  ),
  BIC = sapply(
    candidate_models,
    BIC
  ),
  training_RMSE = sapply(
    candidate_models,
    function(model) {
      sqrt(
        mean(
          residuals(model)^2,
          na.rm = TRUE
        )
      )
    }
  )
)

kable(
  candidate_comparison,
  digits = 3,
  caption = "In-sample comparison of candidate AR models"
)
```

Table 2: In-sample comparison of candidate AR models

	model	lag	log_likelihood	AICc	BIC	training_RMSE
AR(2)	AR(2)	2	-221.965	452.300	462.839	1.722
AR(3)	AR(3)	3	-220.802	452.165	465.241	1.704
AR(4)	AR(4)	4	-220.635	454.062	469.634	1.702
AR(5)	AR(5)	5	-218.373	451.812	469.837	1.667

```
lapply(
  candidate_models,
  summary
)
```

```
## $`AR(2)`
## Series: inflation_sa_diff
## ARIMA(2,0,0) with non-zero mean
```

```
##
## Coefficients:
##          ar1      ar2    mean
##       0.1162  -0.4058  0.0049
## s.e.  0.0855   0.0848  0.1262
##
## sigma^2 = 3.048:  log likelihood = -221.96
## AIC=451.93   AICc=452.3   BIC=462.84
##
## Training set error measures:
##                       ME    RMSE      MAE      MPE     MAPE      MASE
## Training set -0.00161321 1.72242 1.041655 48.55522 207.7487 0.5945429
##                     ACF1
## Training set 0.05935156
##
## $`AR(3)`
## Series: inflation_sa_diff
## ARIMA(3,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3    mean
##       0.1755  -0.4227  0.1412  0.0053
## s.e.  0.0930   0.0846  0.0920  0.1452
##
## sigma^2 = 3.011:  log likelihood = -220.8
## AIC=451.6   AICc=452.17   BIC=465.24
##
## Training set error measures:
##                        ME     RMSE      MAE      MPE     MAPE      MASE
## Training set -0.002282791 1.704326 1.063918 45.73558 228.3796 0.6072495
##                      ACF1
## Training set -0.006725258
##
## $`AR(4)`
## Series: inflation_sa_diff
## ARIMA(4,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3     ar4    mean
##       0.1675  -0.3993  0.1314  0.0536  0.0051
## s.e.  0.0939   0.0936  0.0934  0.0926  0.1530
##
## sigma^2 = 3.03:  log likelihood = -220.63
## AIC=453.27   AICc=454.06   BIC=469.63
##
## Training set error measures:
##                        ME     RMSE      MAE      MPE     MAPE      MASE
## Training set -0.002245887 1.701733 1.052995 45.08096 217.1798 0.6010152
##                     ACF1
## Training set 0.01187622
##
## $`AR(5)`
## Series: inflation_sa_diff
## ARIMA(5,0,0) with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3     ar4      ar5    mean
```

```
##       0.1785  -0.3722  0.0508  0.0876  -0.1945  0.0060
## s.e.  0.0920   0.0924  0.0988  0.0920   0.0904  0.1264
##
## sigma^2 = 2.933:  log likelihood = -218.37
## AIC=450.75   AICc=451.81   BIC=469.84
##
## Training set error measures:
##                       ME     RMSE      MAE      MPE     MAPE      MASE
## Training set -0.001930741 1.666618 1.027375 47.95667 194.2328 0.5863919
##                      ACF1
## Training set 0.006863363
```

The training RMSE measures in-sample fit. A rolling-window evaluation is used below to assess genuine out-of-sample forecasting performance.

9 Rolling-window forecast evaluation

The candidate models are compared using repeated one-month-ahead forecasts.

At each forecast origin:

1. The model uses only the most recent 60 months.
2. The STL decomposition is re-estimated from that training window.
3. An ARIMA$(p, 1, 0)$ model is fitted to seasonally adjusted inflation.
4. The seasonally adjusted forecast is calculated.
5. The projected seasonal component is added back.
6. The forecast is compared with the next observed inflation rate.

This evaluates forecasts of actual observed inflation rather than only changes in seasonally adjusted inflation.

9.1 Rolling forecasting function

```
forecast_stl_ar <- function(
  y,
  h,
  p,
  s.window = 13,
  use_drift = TRUE,
  level = c(80, 95),
  ...
) {

  # Estimate seasonality using only the current training window.
  decomposition <- stl(
    y,
    s.window = s.window,
    robust = TRUE
  )

  # Forecast the seasonally adjusted series.
  forecast_adjusted <- function(
    x,
    h,
    level = c(80, 95),
    ...
  ) {

    fit <- Arima(
```

```
      x,
      order = c(p, 1, 0),
      include.drift = use_drift,
      method = "ML"
    )

    forecast(
      fit,
      h = h,
      level = level
    )
  }

  # Forecast adjusted inflation and add seasonality back.
  forecast(
    decomposition,
    forecastfunction = forecast_adjusted,
    h = h,
    level = level
  )
}
```

9.2 Generate rolling forecast errors

```
rolling_errors <- lapply(
  candidate_lags,
  function(p) {
    tsCV(
      y = inflation_ts,
      forecastfunction = forecast_stl_ar,
      h = cv_horizon,
      window = window_size,
      p = p,
      s.window = seasonal_window,
      use_drift = use_drift
    )
  }
)

names(rolling_errors) <- paste0(
  "AR(",
  candidate_lags,
  ")"
)
```

9.3 Calculate rolling forecast accuracy

```
rolling_results <- bind_rows(
  lapply(
    seq_along(candidate_lags),
    function(i) {

      errors <- as.numeric(
        rolling_errors[[i]]
      )
```

```
      errors <- errors[
        is.finite(errors)
      ]

      data.frame(
        model = paste0(
          "AR(",
          candidate_lags[i],
          ")"
        ),
        lag = candidate_lags[i],
        forecasts = length(errors),
        ME = mean(errors),
        RMSE = sqrt(
          mean(errors^2)
        ),
        MAE = mean(
          abs(errors)
        )
      )
    }
  )
) %>%
  arrange(
    RMSE,
    lag
  )

kable(
  rolling_results,
  digits = 3,
  caption = paste(
    "Rolling-window one-month-ahead",
    "forecast accuracy for observed inflation"
  )
)
```

Table 3: Rolling-window one-month-ahead forecast accuracy for observed inflation

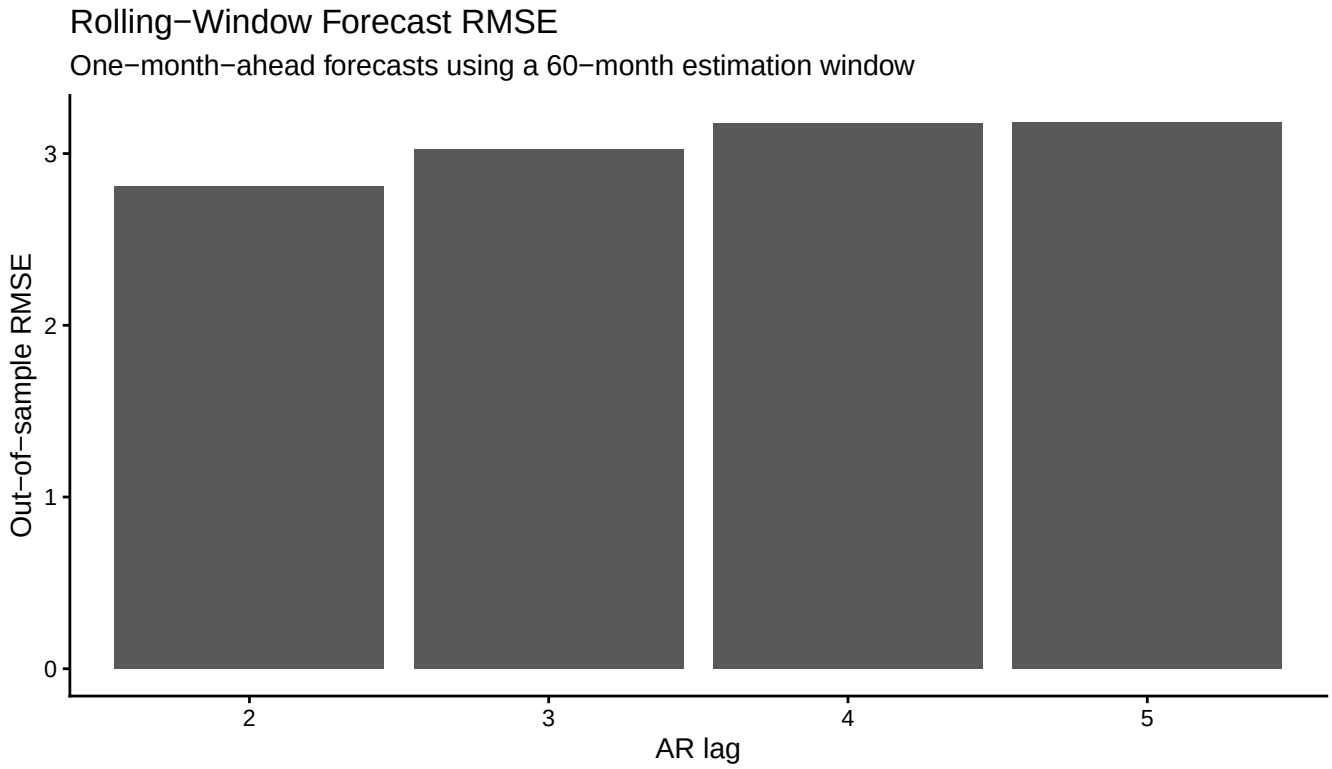
model	lag	forecasts	ME	RMSE	MAE
AR(2)	2	54	0.034	2.807	1.612
AR(3)	3	54	0.023	3.027	1.728
AR(4)	4	54	-0.204	3.176	1.844
AR(5)	5	54	-0.274	3.180	1.830

The rolling RMSE and MAE are measured in percentage points of observed monthly inflation.

9.4 Plot rolling-window RMSE

```
ggplot(
  rolling_results,
  aes(
    x = factor(lag),
    y = RMSE
```

```
  )
) +
  geom_col() +
  labs(
    title = "Rolling-Window Forecast RMSE",
    subtitle = paste0(
      "One-month-ahead forecasts using a ",
      window_size,
      "-month estimation window"
    ),
    x = "AR lag",
    y = "Out-of-sample RMSE"
  ) +
  theme_classic()
```



9.5 Select the preferred lag

```
selected_p <- rolling_results$lag[1]

selected_p
```

```
## [1] 2
```

The rolling-window evaluation selects an **AR(2)** model.

10 Final model

An AR(2) model for the first difference of seasonally adjusted inflation is equivalent to an ARIMA(2,1,0) model for seasonally adjusted inflation in levels.

The final model is estimated using all available observations.

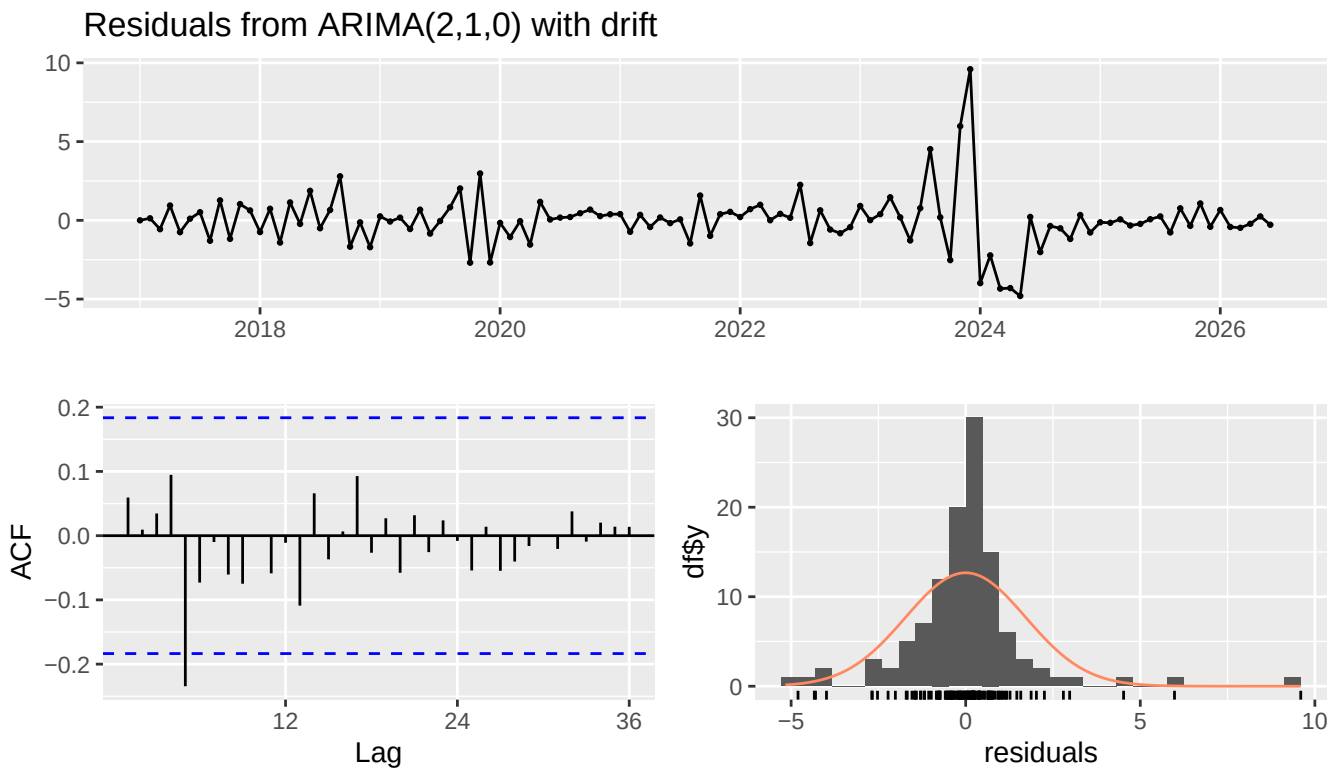
```r
final_model <- Arima(
  inflation_sa,
  order = c(selected_p, 1, 0),
  include.drift = use_drift,
  method = "ML"
)

summary(final_model)
```

```
## Series: inflation_sa
## ARIMA(2,1,0) with drift
##
## Coefficients:
##          ar1      ar2   drift
##       0.1162  -0.4058  0.0049
## s.e.  0.0855   0.0848  0.1262
##
## sigma^2 = 3.048:  log likelihood = -221.96
## AIC=451.93   AICc=452.3   BIC=462.84
##
## Training set error measures:
##                       ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -0.001582818 1.714849 1.032534 -5.294218 25.96902 0.3675608
##                    ACF1
## Training set 0.0593529
```

10.1 Residual diagnostics

```r
checkresiduals(final_model)
```




```
##
```

```
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,1,0) with drift
## Q* = 15.067, df = 21, p-value = 0.8196
##
## Model df: 2.   Total lags used: 23
```

The residual ACF should contain no important remaining autocorrelation, and the Ljung–Box test should ideally have a p-value greater than 0.05.

11 Forecast the next four months

11.1 Forecasting function for the selected model

```
forecast_selected_model <- function(
  y,
  h,
  level = c(80, 95),
  ...
) {

  fit <- Arima(
    y,
    order = c(selected_p, 1, 0),
    include.drift = use_drift,
    method = "ML"
  )

  forecast(
    fit,
    h = h,
    level = level
  )
}
```

11.2 Produce the forecast

```
inflation_forecast <- forecast(
  inflation_stl,
  forecastfunction = forecast_selected_model,
  h = final_horizon,
  level = c(80, 95)
)

inflation_forecast
```

```
##          Point Forecast      Lo 80    Hi 80     Lo 95    Hi 95
## Jul 2026       1.591385 -0.6458834 3.828653 -1.830222 5.012991
## Aug 2026       2.408654 -0.9441770 5.761485 -2.719058 7.536366
## Sep 2026       2.187647 -1.5358529 5.911148 -3.506955 7.882250
## Oct 2026       2.356862 -1.6254086 6.339133 -3.733495 8.447219
```

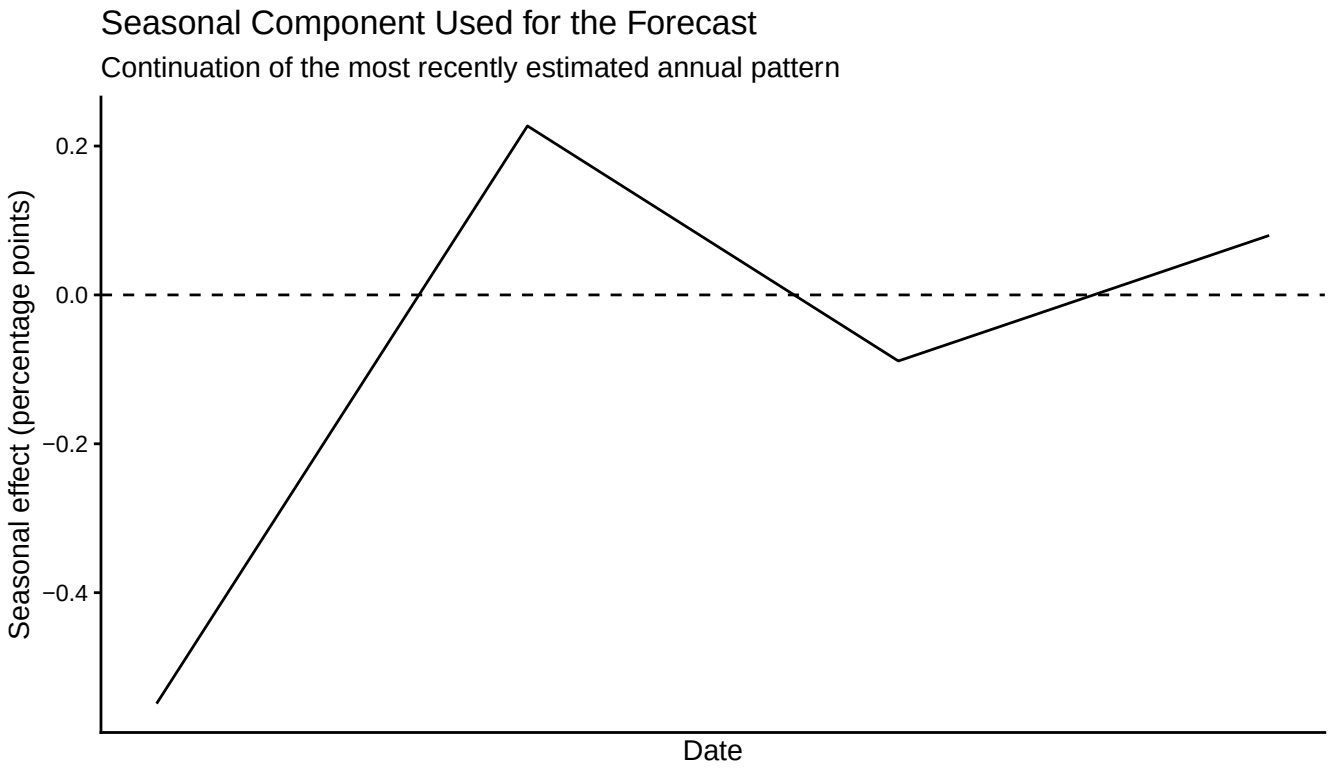
This forecast is for observed monthly inflation. The STL procedure forecasts the seasonally adjusted inflation rate and then adds the projected seasonal component back.

12 Seasonal component used for the forecast

The most recently estimated annual seasonal pattern is repeated into the forecast period.

```
future_seasonal <- ts(
  rep(
    tail(
      seasonal_ts,
      frequency(inflation_ts)
    ),
    length.out = final_horizon
  ),
  start = tsp(inflation_ts)[2] +
    1 / frequency(inflation_ts),
  frequency = frequency(inflation_ts)
)

autoplot(future_seasonal) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
    title = "Seasonal Component Used for the Forecast",
    subtitle = "Continuation of the most recently estimated annual pattern",
    x = "Date",
    y = "Seasonal effect (percentage points)"
  ) +
  theme_classic()
```



13 Create the final CSV output

The final CSV contains:

- the last 12 observed months;
- the next 4 forecast months;
- point forecasts;
- seasonally adjusted values;
- seasonal effects;
- 80% prediction intervals;
- 95% prediction intervals.

13.1 Last 12 observed months

```
last_12_observed <- data.frame(
  date = tail(
    inflation$date,
    12
  ),
  period = "Observed",
  actual_inflation_pct = tail(
    inflation$monthly_inflation_pct,
    12
  ),
  forecast_inflation_pct = NA_real_,
  seasonally_adjusted_value_pct = as.numeric(
    tail(
      inflation_sa,
      12
    )
  ),
  seasonal_component_pp = as.numeric(
    tail(
      seasonal_ts,
      12
    )
  ),
  lower_80 = NA_real_,
  upper_80 = NA_real_,
  lower_95 = NA_real_,
  upper_95 = NA_real_
)
```

13.2 Next four forecast months

```
forecast_dates <- seq.Date(
  from = max(inflation$date),
  by = "month",
  length.out = final_horizon + 1
)[-1]

forecast_point_values <- as.numeric(
  inflation_forecast$mean
)

future_seasonal_values <- as.numeric(
  future_seasonal
```

```
)

next_4_forecasts <- data.frame(
  date = forecast_dates,
  period = "Forecast",
  actual_inflation_pct = NA_real_,
  forecast_inflation_pct =
    forecast_point_values,
  seasonally_adjusted_value_pct =
    forecast_point_values -
    future_seasonal_values,
  seasonal_component_pp =
    future_seasonal_values,
  lower_80 = as.numeric(
    inflation_forecast$lower[, 1]
  ),
  upper_80 = as.numeric(
    inflation_forecast$upper[, 1]
  ),
  lower_95 = as.numeric(
    inflation_forecast$lower[, 2]
  ),
  upper_95 = as.numeric(
    inflation_forecast$upper[, 2]
  )
)
```

13.3 Combine observed and forecast observations

```
csv_output <- bind_rows(
  last_12_observed,
  next_4_forecasts
)

if (nrow(csv_output) != 16) {
  stop(
    "The final CSV output should contain exactly 16 rows."
  )
}

kable(
  csv_output,
  digits = 3,
  caption = paste(
    "Last 12 observed months and",
    "next 4 monthly inflation forecasts"
  )
)
```

Table 4: Last 12 observed months and next 4 monthly inflation forecasts

date	period	actual_inflation_pct	forecast_inflation_pct	seasonally_adjusted_value_pct	seasonal_component_pp	lower_80	upper_80	lower_95	upper_95
2025-07-01	Observed	1.9	NA	2.449	-0.549	NA	NA	NA	NA

date	period	actual_inflation	forecast_inflation	seasonally_adjusted_value	seasonal_pct	component	lower_80	upper_80	lower_95	upper_95
2025-08-01	Observed	1.9	NA	1.673	0.227	NA	NA	NA	NA	
2025-09-01	Observed	2.1	NA	2.189	-0.089	NA	NA	NA	NA	
2025-10-01	Observed	2.3	NA	2.220	0.080	NA	NA	NA	NA	
2025-11-01	Observed	2.5	NA	3.094	-0.594	NA	NA	NA	NA	
2025-12-01	Observed	2.8	NA	2.780	0.020	NA	NA	NA	NA	
2026-01-01	Observed	2.9	NA	3.053	-0.153	NA	NA	NA	NA	
2026-02-01	Observed	2.9	NA	2.800	0.100	NA	NA	NA	NA	
2026-03-01	Observed	3.4	NA	2.191	1.209	NA	NA	NA	NA	
2026-04-01	Observed	2.6	NA	2.010	0.590	NA	NA	NA	NA	
2026-05-01	Observed	2.1	NA	2.492	-0.392	NA	NA	NA	NA	
2026-06-01	Observed	1.9	NA	2.347	-0.447	NA	NA	NA	NA	
2026-07-01	Forecast	NA	1.591	2.140	-0.549	-0.646	3.829	-1.830	5.013	
2026-08-01	Forecast	NA	2.409	2.182	0.227	-0.944	5.761	-2.719	7.536	
2026-09-01	Forecast	NA	2.188	2.276	-0.089	-1.536	5.911	-3.507	7.882	
2026-10-01	Forecast	NA	2.357	2.277	0.080	-1.625	6.339	-3.733	8.447	

13.4 Export the CSV

```
dir.create(
  dirname(output_path),
  recursive = TRUE,
  showWarnings = FALSE
)

write_csv(
  csv_output,
  output_path,
  na = ""
)

cat(
  "The CSV file was created successfully at:\n",
  output_path,
  "\n"
)
```

```
## The CSV file was created successfully at:
##  C:/Users/narag/Desktop/my little project/Inflation Prediction ARG/files/argentina_inflation_forecast.cs
```

14 Plot the final forecast

```
autoplot(inflation_forecast) +
  labs(
    title = "Forecast of Monthly Inflation in Argentina",
    subtitle = paste0(
      "Four-month forecast using STL and ARIMA(",
      selected_p,
      ",1,0)"
    ),
    x = "Date",
    y = "Monthly inflation (%)"
  ) +
  theme_classic()
```